

FutureKawa

Guide d'Installation & Démarrage

Équipe : Rudy · Florian · Noé · Josselin

Version : 1.0

Date : 05/07/2026

Projet : FutureKawa

Sommaire

Guide d'Installation & Démarrage — FutureKawa

01 — Vue d'ensemble & prérequis

02 — Configuration (`.env`)

03 — Démarrage rapide

04 — Référence des commandes Make

05 — Services & ports

06 — Dépannage

Guide d'Installation & Démarrage — FutureKawa

Projet	FutureKawa
Livrable	Guide d'installation & démarrage
Périmètre	Mise en route complète de la stack (siège, exploitations, IoT, services)
Outils	Docker + Docker Compose + Make
Équipe	Rudy · Florian · Noé · Josselin

Sommaire

Section	Fichier
Vue d'ensemble & prérequis	01-prerequis.md
Configuration (<code>.env</code>)	02-configuration.md
Démarrage rapide	03-demarrage-rapide.md
Référence des commandes Make	04-commandes-make.md
Services & ports	05-services-ports.md
Dépannage	06-depannage.md

En trois commandes

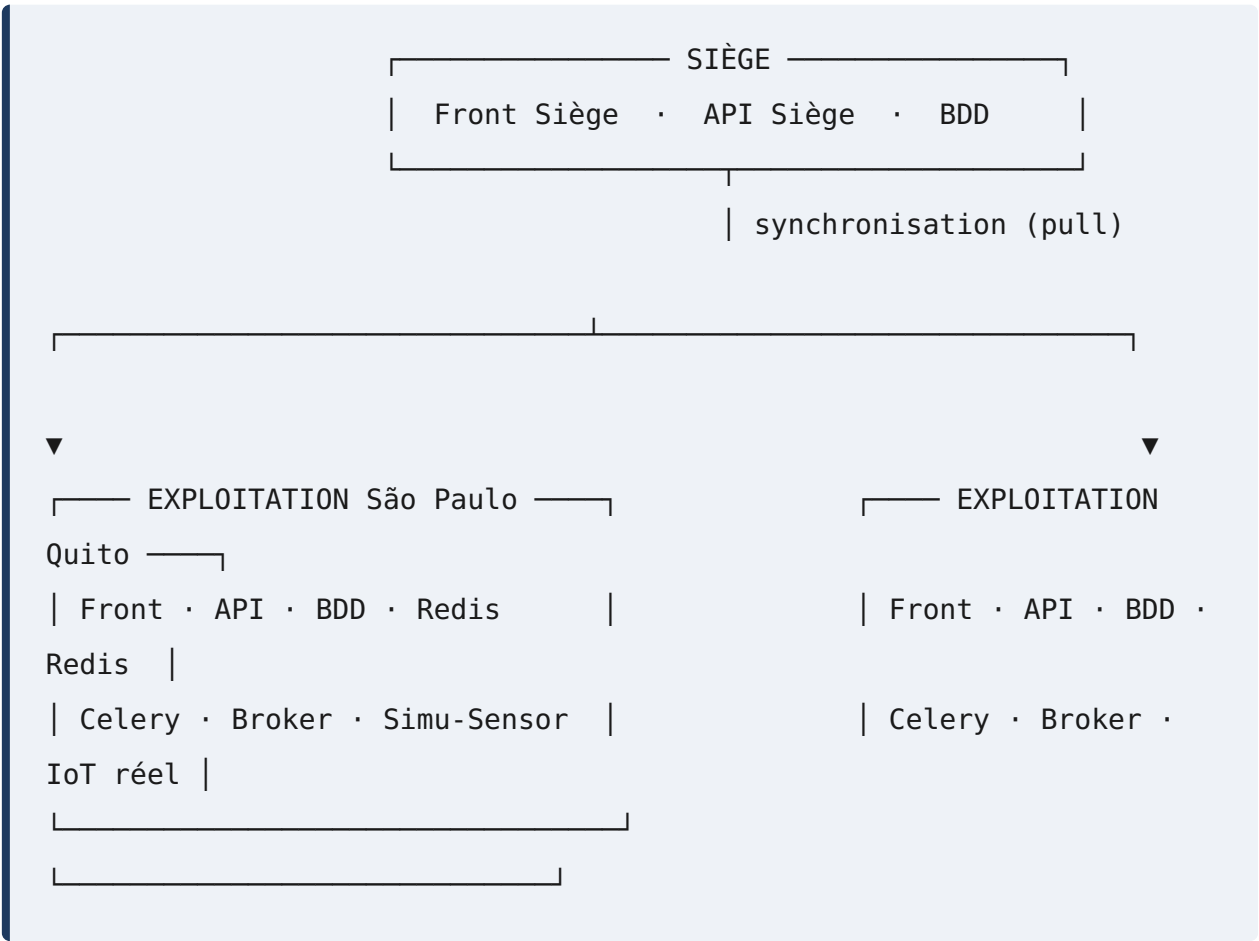
```
git clone <url-du-depot> && cd futurekawa
cp .env.example .env
make demo
```

`make demo` reconstruit et démarre la plantation São Paulo (avec simulateur de capteurs) + le siège, puis charge des données de démonstration. Tout est accessible après ~30 s.

01 — Vue d'ensemble & prérequis

Ce qui va être installé

FutureKawa est un système **conteneurisé** : tout tourne dans Docker. Il n'y a donc **rien à installer manuellement** (ni Python, ni Node, ni PostgreSQL) — Docker s'occupe de tout.



Prérequis

Outil	Rôle	Vérification
Docker	Exécution des conteneurs	<code>docker --version</code>
Docker Compose	Orchestration multi-conteneurs	<code>docker compose version</code>
Make	Raccourcis de commandes	<code>make --version</code>

Outil	Rôle	Vérification
Git	Récupération du code	<code>git --version</code>

Optionnel : **PlatformIO / Arduino** uniquement si l'on programme un vrai ESP32 (voir livrable IoT). Le simulateur de capteurs ne le nécessite pas.

Ressources conseillées

Ressource	Minimum
RAM	4 Go (8 Go confortable — plusieurs bases + Redis + workers)
Disque	~5 Go pour les images Docker et les volumes
OS	Linux, macOS, ou Windows (WSL2)

Récupérer le projet

```
git clone <url-du-depot>
cd futurekawa
```

Structure utile à connaître :

```
futurekawa/
├─ docker-compose.exploit.yml    # stack exploitation (São Paulo)
├─ docker-compose.exploit-1.yml  # 2° exploitation (Quito)
├─ docker-compose.siege.yml      # stack siège
├─ docker-compose.ci.yml         # environnement CI
├─ Makefile                      # commandes (make demo, down, ...)
├─ .env.example                  # modèle de configuration
├─ API-exploitation/  API-siege/ # backends Django
├─ front-exploitation/ front-siege/ # frontends Vue
└─ livrables/                   # cette documentation
```

Les deux façons de démarrer

Commande	Ce qu'elle lance
<code>make demo</code>	São Paulo (avec simulateur) + Siège — recommandé
<code>make full</code>	São Paulo + Quito + Siège (stack complète multi-sites)

Le détail est dans [Démarrage rapide](#).

02 — Configuration (`.env`)

Créer le fichier `.env`

Toute la configuration passe par un fichier `.env` à la racine, copié depuis le modèle fourni :

```
cp .env.example .env
```

Pour un démarrage en **démo local**, les valeurs par défaut suffisent : on peut lancer `make demo` sans rien modifier.

Les blocs de configuration

Base de données

```
POSTGRES_USER=admin
POSTGRES_PASSWORD=admin
POSTGRES_DB_SIEGE=futureKawa-bdd-siege
POSTGRES_DB_EXPLOITATION=futureKawa-bdd-exploitation
```

Comptes admin (créés automatiquement au 1^{er} démarrage)

```
ADMIN_PASSWORD=admin
ADMIN_EMAIL=admin@futurekawa.com
```

MQTT (broker + clients navigateur)

```
MQTT_USER=adminFutureKawa
MQTT_PASSWORD=admin
MQTT_BROKER=broker
MQTT_PORT=1883
VITE_MQTT_BROKER_URL=ws://localhost:9001
```



```
VITE_MQTT_USER=adminFutureKawa  
VITE_MQTT_PASSWORD=admin
```

Les variables `MQTT_*` (broker) et `VITE_MQTT_*` (front) doivent rester **cohérentes** : un mot de passe désynchronisé empêche le front de se connecter au broker.

Email (Gmail) — pour les alertes

```
EMAIL_HOST=smtp.gmail.com  
EMAIL_PORT=587  
EMAIL_USE_TLS=True  
EMAIL_HOST_USER=noreplyfuturekawa@gmail.com  
EMAIL_HOST_PASSWORD=<mot-de-passe-application-16-caracteres>  
DEFAULT_FROM_EMAIL=noreply@futurekawa.com
```

Gmail exige un **mot de passe d'application** (pas le mot de passe du compte) : <https://myaccount.google.com/apppasswords>

Celery & Redis

```
CELERY_BROKER_URL=redis://redis:6379/0  
CELERY_RESULT_BACKEND=redis://redis:6379/0  
REDIS_URL=redis://redis:6379/0
```

Django

```
DJANGO_SETTINGS_MODULE=core.settings.local  
SECRET_KEY=your-secret-key-here-change-in-production  
ENV=dev # dev | prod | test
```

Variables sensibles en production

Variable	En dev	En production
<code>SECRET_KEY</code>	valeur par défaut	à changer (aléatoire, secrète)
<code>POSTGRES_PASSWORD</code>	<code>admin</code>	mot de passe fort
<code>ADMIN_PASSWORD</code>	<code>admin</code>	à changer
<code>EMAIL_HOST_PASSWORD</code>	mot de passe d'app	idem, jamais commité
<code>SITE_TOKEN_ENCRYPTION_KEY</code> (siège)	générée	à sauvegarder à part (chiffre les tokens des sites)

Le fichier `.env` **ne doit jamais être commité** (il est ignoré par Git). Seul `.env.example` est versionné. Détails dans le livrable [Sécurité](#).

Piège connu : les doublons

Ne pas redéfinir deux fois la même variable dans `.env` : la **dernière occurrence gagne** et peut désynchroniser le broker de ses clients. Le modèle `.env.example` documente ce point en commentaire.

03 — Démarrage rapide

La commande unique

```
make demo
```

Cette seule commande enchaîne automatiquement :

1. Reset complet (arrêt + suppression des volumes existants)
2. Build + démarrage de l'exploitation São Paulo (profil demo, simu-sensor actif)
3. Build + démarrage du siège
4. Attente que les deux API répondent (manage.py check)
5. Chargement des données de démo (seed_demo_data)
6. Affichage de l'état et des URLs (status)

Après ~30 secondes, la stack est prête.

Les URLs d'accès

Interface	URL
Front Siège (supervision globale)	http://localhost:8180
Front Exploitation (données plantation)	http://localhost:8182
Simu-Sensor (capteur virtuel)	http://localhost:8181
API Exploitation	http://localhost:8600/api
API Siège	http://localhost:8500/api
Broker MQTT	localhost:1883

Se connecter

Des comptes de démonstration sont créés automatiquement. Voir la variable `ADMIN_EMAIL` / `ADMIN_PASSWORD` du `.env` :

```
Email      : admin@futurekawa.com
Mot de passe : admin
```

Ce sont des identifiants de **démonstration** — à changer impérativement en production.

Charger / recharger les données de démo

```
make seed    # recharge les données de démo (stack déjà lancée)
```

`seed` exécute `python manage.py seed_demo_data` dans l'API exploitation **et** l'API siège : entrepôts, capteurs, lots, mesures réalistes, plus les sites côté siège.

Stack complète multi-sites

Pour démarrer les **deux** plantations (São Paulo + Quito) et le siège :

```
make full
```

C'est le mode le plus proche d'un déploiement réel, avec un siège qui agrège plusieurs exploitations.

Vérifier que tout tourne

```
make status
```

Affiche l'état (`up` / `down`) de chaque conteneur et les URLs. Exemple :

Front Siège	<code>http://localhost:8180</code>	Oui up
API Siège	<code>http://localhost:8500/api</code>	Oui up

Front Exploitation	http://localhost:8182	Oui up
API Exploitation	http://localhost:8600/api	Oui up
Simu Sensor	http://localhost:8181	Oui up
MQTT Broker	localhost:1883	Oui up

Arrêter

```
make down    # arrête les services (conserve les données)
make clean   # arrête ET supprime les volumes (reset complet)
```

Premier parcours conseillé

1. make demo
2. Ouvrir le Front Exploitation (8182) → voir les capteurs en temps réel
3. Observer le simu-sensor (8181) qui pousse des mesures via MQTT
4. Ouvrir le Front Siège (8180) → synchroniser le site et voir les données consolidées

04 — Référence des commandes

Make

Aide intégrée

```
make help    # liste toutes les commandes disponibles
```

Stack

Commande	Effet
<code>make demo</code>	Reset + démarre São Paulo (simu-sensor) + Siège + seed — recommandé
<code>make full</code>	Démarre São Paulo + Quito + Siège (stack complète)
<code>make seed</code>	Recharge les données de démo (stack déjà lancée)
<code>make down</code>	Arrête tous les services (conserve les volumes)
<code>make clean</code>	Arrête + supprime les volumes (reset complet)
<code>make rebuild</code>	Reconstruit toutes les images sans cache

Monitoring

Commande	Effet
<code>make status</code>	État des conteneurs + URLs d'accès
<code>make logs</code>	Logs temps réel de tous les services
<code>make logs-broker</code>	Logs du broker MQTT uniquement

Tests

Commande	Effet
<code>make test</code>	Lance tous les tests (exploitation + siège + simu)
<code>make test-api-exploitation</code>	Tests Django de l'API Exploitation
<code>make test-api-siege</code>	Tests Django de l'API Siège
<code>make test-simu-sensor</code>	Tests Vitest du simulateur
<code>make test-simu-sensor-e2e</code>	Tests end-to-end du simulateur

IoT (ESP32)

Commande	Effet
<code>make iot-all</code>	Install + setup + build + upload (tout en un)
<code>make iot-dev</code>	Compile et téléverse sur l'ESP32
<code>make iot-monitor</code>	Logs série en direct
<code>make iot-find-port</code>	Détecte le port de l'ESP32
<code>make iot-help</code>	Aide détaillée IoT

Le détail IoT (matériel, câblage, code Arduino) est dans le livrable **IoT**.

Sous le capot

Les commandes Make encapsulent des appels `docker compose`. Par exemple :

```
demo:
  $(EXPLOIT) --profile demo down -v --remove-orphans
  $(EXPLOIT) --profile demo up -d --build          # docker compose -f
  docker-compose.exploit.yml
  $(SIEGE) up -d --build                          # docker compose -f
```

```
docker-compose.siege.yml
... attente des API ...
make seed
make status
```

Raccourci Makefile

Équivaut à

`$(EXPLOIT)`

```
docker compose -f docker-
compose.exploit.yml
```

`$(SIEGE)`

```
docker compose -f docker-
compose.siege.yml
```

On peut donc toujours agir directement en *docker compose* si besoin d'un contrôle plus fin (un seul service, un *restart* , etc.).

05 — Services & ports

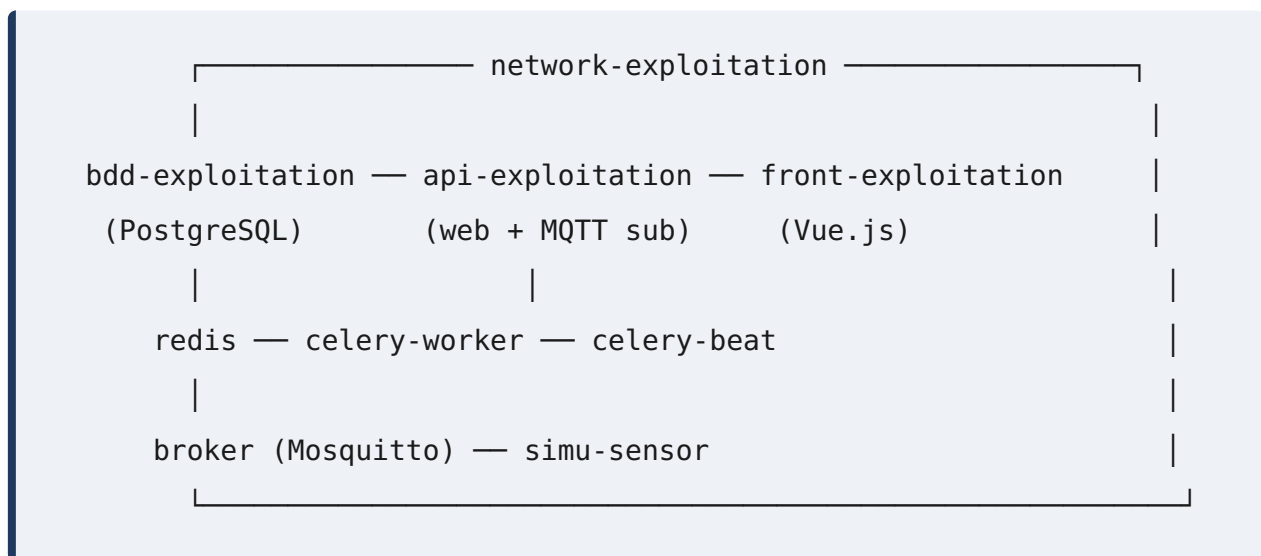
Tableau des services

Service	Technologie	Port	Conteneur
Front Siège	Vue 3 + Vite	8180	futurekawa-front-siege-dev
API Siège	Django REST + PostgreSQL	8500	futurekawa-api-siege-dev
Front Exploitation	Vue 3 + Vite	8182	futurekawa-front-exploitation-dev
API Exploitation	Django REST + PostgreSQL	8600	futurekawa-api-exploitation-dev
Simu-Sensor	Vue 3 (mode démo)	8181	futurekawa-simu-sensor-dev
Broker MQTT	Mosquitto 2.x	1883	futurekawa-broker-dev
Redis	Redis 7	6379	futurekawa-redis-dev
Celery Worker	Celery 5	—	futurekawa-celery-worker-dev
Celery Beat	Celery 5	—	futurekawa-celery-beat-dev
BDD Siège / Exploitation	PostgreSQL 16	interne	futurekawa-bdd-*-dev

Les fichiers docker-compose

Fichier	Rôle
<code>docker-compose.exploit.yml</code>	Exploitation São Paulo (API, BDD, Redis, Celery, broker, front, simu)
<code>docker-compose.exploit-1.yml</code>	2 ^e exploitation (Quito, IoT réel)
<code>docker-compose.siege.yml</code>	Siège (API, BDD, front)
<code>docker-compose.ci.yml</code>	Environnement d'intégration continue

Cartographie des conteneurs d'une exploitation



Le **siège** est raccordé aux réseaux des exploitations pour pouvoir les interroger.

Volumes (données persistantes)

Volume	Contenu
<code>bdd_siege_data</code>	Base PostgreSQL du siège
<code>bdd_exploitation_data</code>	Base PostgreSQL de l'exploitation
<code>redis_data</code>	Données Redis (transitoires)
<code>broker_data</code>	Données du broker
<code>broker_certs</code>	Certificats TLS du broker

`make clean` supprime ces volumes → perte des données. À réserver au développement.

Modifier un port

Les ports sont définis dans les fichiers `docker-compose*.yaml` (section `ports`). Exemple :

```
front-exploitation:
  ports:
    - "8182:80"      # <port-hôte>:<port-conteneur>
```

Changer le **port hôte** (à gauche) suffit si le port est déjà pris sur la machine.

06 — Dépannage

Méthode générale

```
make status          # quels services sont down / unhealthy ?
make logs            # logs agrégés
docker logs <conteneur> # logs d'un service précis
```

Problèmes fréquents

Un port est déjà utilisé

```
Error: bind: address already in use
```

Cause	Solution
Un autre service occupe le port (8180, 8500, 8600, 1883...)	Arrêter ce service, ou changer le port hôte dans <code>docker-compose*.yaml</code>

Les conteneurs ne démarrent pas / redémarrent en boucle

```
docker logs futurekawa-api-exploitation-dev # lire l'erreur exacte
```

Cause probable	Solution
<code>.env</code> absent	<code>cp .env.example .env</code>
Base pas prête au démarrage	Les healthchecks gèrent l'ordre ; réessayer <code>make demo</code>
Image obsolète	<code>make rebuild</code>

Le front ne reçoit pas les mesures temps réel

Cause	Solution
<code>VITE_MQTT_*</code> $\neq$ <code>MQTT_*</code>	Aligner les identifiants MQTT dans <code>.env</code>

Cause	Solution
Broker down	<code>make logs-broker</code> , redémarrer le broker
Mauvaise URL WebSocket	Vérifier <code>VITE_MQTT_BROKER_URL=ws://localhost:9001</code>

Les emails d'alerte ne partent pas

Cause	Solution
Worker Celery arrêté	<code>docker logs futurekawa-celery-worker-dev</code> ; redémarrer
File Celery qui gonfle	<code>docker exec -it futurekawa-redis-dev redis-cli LLEN celery</code>
SMTP mal configuré	Vérifier <code>EMAIL_HOST_PASSWORD</code> (mot de passe d'application Gmail)

Détails dans les livrables [Celery](#) et [Redis](#).

Le siège n'affiche pas les données d'un site

Cause	Solution
Site jamais synchronisé	Lancer la synchronisation depuis le front siège
Token invalide	Vérifier le token du site (voir livrable Sécurité)
Exploitation injoignable	Vérifier que l'API exploitation répond (<code>/api/health/</code>)

Repartir de zéro

```
make clean      # supprime conteneurs + volumes ( perte des données)
make demo       # reconstruit tout proprement
```

Vérifications de santé

```
curl -s http://localhost:8600/api/health/ # API exploitation
curl -s http://localhost:8500/api/health/ # API siège
```

Une réponse `200 OK` confirme que l'API est opérationnelle.

Aller plus loin

Besoin	Livrable
Sauvegardes, reprise après panne	Maintenance
Endpoints de l'API	API
Authentification, tokens, secrets	Sécurité
Broker MQTT	Broker
IoT / ESP32	IoT